

Курсовая работа по курсу дискретного анализа: Алгоритм LZ77

Выполнил студент группы 08-303 МАИ *Арусланов Кирилл*.

Условие

Реализовать алгоритм LZ77. По условию требуется сжимать и восстанавливать текст, состоящий только из малых латинских букв. Поиск совпадений производится не по скользящему окну, а по всему уже просмотренному ранее тексту. Формат ввода/вывода:

- Команда **compress** и далее текст — вывести последовательность троек $\langle \text{offset}, \text{len}, \text{char} \rangle$, по одной тройке на строку. Если на последней тройке символ отсутствует, не вывести его.
- Команда **decompress** и далее тройки — восстановить исходный текст. Тройка может не содержать символ (это означает конец текста).

Метод решения

В работе реализован классический подход поиска наибольшего совпадения с ранее просмотренной частью строки, но для эффективного поиска используется суффиксный массив (Suffix Array) и массив длин общих префиксов (LCP). Основные идеи:

1. Строится преобразованная строка с явным терминатором минимального кода (функция `map_with_terminator`). Это делает все суффиксы сравнимыми и корректно обрабатывает границы, также сдвиг способствует уменьшению затрат по памяти при сортировке подсчетом.
2. Для строки строится суффиксный массив методом сортировки по классам ($O(n \log n)$ в практике) — функция `build_sa_mapped`.
3. На основании суффиксного массива рассчитывается LCP (массив длин общих префиксов) — функция `build_lcp_mapped`. Это даёт возможность быстро оценивать длину совпадения между текущей позицией и любой позицией в предыдущей части текста.
4. Для каждой позиции *pos* берутся соседи по SA (в обе стороны) и по массиву LCP ищется наибольшее вхождение, которое начинается строго раньше *pos* (т.е. в ранее просмотренной части). Если наибольшая длина равна нулю, кодируется одиночный символ как $0, 0, c$; иначе кодируется тройка *offset, poslen, nextchar* или, если совпадение выходит до конца текста, тройка без символа.

Сложности по времени и памяти:

- Построение SA — около $O(n \log n)$ по времени и $O(n)$ по памяти (включая строки и массивы). Для практических размеров (до нескольких сотен тысяч символов) работает эффективно.
- Вычисление LCP — $O(n)$.
- Проход по позициям и выбор лучшего вхождения — в худшем случае может быть до $O(n^2)$ при крайне неблагоприятной структуре и если поиск соседей долго просматривает, но на реальных данных благодаря LCP обычно быстрее.

Описание программы

Программа реализована в одной точке входа (файл с кодом `main.cpp`). Основные функции/компоненты:

- `map_with_terminator(const std::string&)` — сопоставляет символы строки смещёнными кодами и добавляет терминатор (байт 0), чтобы суффиксы сравнивались корректно.
- `build_sa_mapped(const std::string&)` — строит суффиксный массив методом сортировки по классам (алгоритм типа *prefix doubling*). Возвращает вектор позиций суффиксов.
- `build_lcp_mapped(const std::string&, const std::vector<int>&sa)` — строит LCP во времени $O(n)$ по алгоритму Касаи.
- `lz77_compress_sa(const std::string&)` — основная процедура сжатия: для каждой позиции находит лучшее совпадение в предыдущей части с помощью SA/LCP и формирует вектор троек (структура `Token`).
- `lz77_decompress(const std::vector<Token>&)` — выполняет восстановление строки из набора троек, корректно поддерживает перекрывающиеся копии.
- `parse_tokens_from_stream` (в улучшенной версии) — парсит входные строки с тройками построчно, устойчив к лишним пробелам.
- `main` — парсит команду `compress/decompress` в одном из двух форматов (команда и данные либо в одной строке, либо на следующей) и вызывает соответствующие функции.

Дневник отладки

Работа была реализована и протестирована на наборе тестов, обеспечивающих корректность и граничные случаи. Краткая хронология и замечания:

- Реализация основных алгоритмов SA и LCP. На этапе раннего тестирования проверялась корректность на простых строках: "a", "aaaaa", "abababab".

- Доработана обработка пробелов при декодировании.
- Скорректированы границы цикла при кодировании.

Тест производительности

Измерения времени работы производились для случайно сгенерированных строк различной длины. Время — wall-clock (мс), замеры одного прогона на каждой длине. В таблице приведено время сжатия, время распаковки и число полученных токенов.

n	compress (ms)	decompress (ms)	tokens
1000	6	0	413
5000	33	0	1698
10000	51	0	3176
30000	140	0	8577
60000	179	0	16080
120000	422	1	30466
240000	939	4	58112
480000	2301	8	110526

Краткий разбор результатов. По таблице видно, что:

- Время распаковки пренебрежимо мало по сравнению со временем сжатия и растёт очень медленно (близко к линейному поведению и с маленькой константой).
- Время сжатия растёт существенно при увеличении n ; это соответствует тому, что доминирующая часть работы — построение суффиксного массива и дополнительные шаги (теоретически близко к $O(n \log n)$ для использованного алгоритма), а также последующая обработка.
- Число токенов растёт примерно линейно с n , при этом отношение токенов к длине строки уменьшается для больших n , что характерно для случайных данных (в относительном выражении вероятность длинных совпадений растёт с объёмом данных).

Выводы

Реализованный алгоритм LZ77 с поиском по всему пройденному тексту и использованием суффиксного массива является корректным и пригоден для задач средней размерности. Применение:

- Архиваторы/компоненты сжатия потока, где требуется найти повторяющиеся подстроки в данных.

- Анализ повторяющихся паттернов в текстах и последовательностях (биоинформатика, сжатие логов и т.п.).

Сложность реализации умеренная: основная сложность связана со стандартными структурами данных (SA/LCP).